

基于 CP 分解与数据感知逐层微调的 CNN 压缩方法研究

矩阵分析课程期末项目报告

2026 年 6 月 17 日

摘要

深度卷积神经网络通常具有较高的参数数量和推理计算量，这给资源受限场景下的部署带来困难。张量分解通过将权重张量重新参数化为低秩因子，为卷积层压缩提供了一种结构化方法。本文研究 CP 分解在 CNN 卷积层压缩中的应用：首先使用 CP-ALS 分解卷积核张量，并将原始卷积层替换为四个轻量卷积层；随后比较直接替换、全局微调、迭代式逐层微调和数据感知逐层微调四种压缩/恢复设置。实验在 MNIST 与 Fashion-MNIST 上进行，模型包括 LeNet-5 与 SmallCNN。结果显示，直接替换后的准确率接近随机；全局微调能够恢复部分性能，但在 Fashion-MNIST 上仍有明显精度损失；迭代式逐层微调显著改善恢复效果，而加入数据感知的层输出重构后，模型在保持 51.44% (LeNet-5) 和 77.50% (SmallCNN) FLOPs 降低的同时，准确率基本恢复到稠密基线模型水平。

关键词：模型压缩；CP 分解；CP-ALS；卷积神经网络；逐层微调

1 引言

近年来，卷积神经网络（CNN）在图像分类任务中取得了稳定而突出的表现，是深度学习方法的重要代表之一 [1]。但模型性能的提升通常伴随更大的参数规模和计算量；对于需要在边缘设备或低功耗环境中部署的模型而言，卷积层带来的存储与推理成本是一个重要限制。模型压缩的目标是在尽量保持预测性能的前提下，降低参数数量和计算复杂度。常见压缩方法包括剪枝、量化、知识蒸馏和低秩分解；其中，张量分解直接利用卷积核权重的多维结构，是一种具有明确数学形式的结构化压缩方法 [2-4]。

CP 分解是基础的张量分解方法之一，它将一个高阶张量近似为若干秩一张量之和 [2]。对于二维卷积层，其权重天然构成一个四阶张量，四个维度分别对应输出通道、输入通道、卷积核高度和卷积核宽度。已有研究将低秩分解用于卷积层加速，并进一步把完整四维卷积核的 CP 分解实现为若干小卷积层的串联结构 [5,6]。若目标秩较小，该结构即可减少参数量与 FLOPs。

这一压缩过程的关键困难在于精度恢复。虽然 CP 分解能够减小权重重构误差，但该目标并不直接保证网络的中间特征和最终分类函数保持不变。直接替换卷积层通常会导致准确率大幅下降；一次性替换所有目标层后进行全局微调虽然简单，但多个层的近似误差会同时传播到后续网络，恢复效果有限。本文因此采用迭代式逐层微调，并进一步引入数据感知的层输出重构：每替换一层后，先使 CP 层在当前特征输入上的输出接近替换前的稠密层输出，再进行分类微调。实验在 MNIST 与 Fashion-MNIST、LeNet-5 与 SmallCNN 上验证该策略的有效性。

2 模型与问题描述

2.1 卷积神经网络层

CNN 中的一个标准二维卷积层由四阶权重张量

$$\mathcal{W} \in \mathbb{R}^{O \times I \times H \times W}$$

定义, 其中 O 和 I 分别为输出和输入通道数, H 和 W 分别为卷积核高度和宽度。忽略偏置时, 该层参数量为 $OIHW$ 。本文的目标是用参数更少的结构重新表示 $\mathcal{W}$, 同时尽量保持原网络的分类性能。

2.2 典范多元 (CP) 分解

CP 分解将一个高阶张量近似表示为多个秩一张量之和。对卷积核张量而言, 其形式为

$$\mathcal{W}[o, i, h, w] \approx \sum_{r=1}^R \lambda_r A[o, r] B[i, r] C[h, r] D[w, r], \quad (1)$$

其中 R 为 CP 秩, A, B, C, D 分别对应输出通道、输入通道、卷积核高度和卷积核宽度四个模态。压缩后参数量约为 $R(O + I + H + W)$ (不含偏置; 精确值为 $R(O + I + H + W) + O$, 偏置项保留在最后一个逐点卷积上)。

2.3 问题描述

按照该分解形式, 原卷积层可替换为四个串联层:

1. 1×1 逐点卷积: $I \rightarrow R$;
2. 深度可分离垂直卷积: $R \rightarrow R$, 卷积核 $H \times 1$;
3. 深度可分离水平卷积: $R \rightarrow R$, 卷积核 $1 \times W$;
4. 1×1 逐点卷积: $R \rightarrow O$ 。

该结构保持输入输出通道数和空间尺寸一致, 同时将单个密集卷积拆解为计算成本更低的结构。

实验使用 MNIST 与 Fashion-MNIST, 输入均为 $1 \times 28 \times 28$, 类别数均为 10。本文考虑两个 CNN:

- **LeNet-5**: 两个卷积层 ($1 \rightarrow 6$ 的 5×5 卷积, $6 \rightarrow 16$ 的 5×5 卷积) 和三个全连接层, 目标秩 $R = 8$ 。
- **SmallCNN**: 三个 3×3 卷积层 ($1 \rightarrow 16 \rightarrow 32 \rightarrow 64$) 和两个全连接层, 目标秩 $R = 16$ 。

3 算法设计

3.1 基于 ALS 的 CP 分解

CP 分解通过最小化 Frobenius 范数下的重构误差求解:

$$\min_{A, B, C, D} \left\| \mathcal{W} - \sum_{r=1}^R \lambda_r a_r \circ b_r \circ c_r \circ d_r \right\|_F^2. \quad (2)$$

该问题非凸。交替最小二乘法 (ALS) 每次固定除一个模态外的所有因子, 将更新转化为线性最小二乘问题。设 $\mathbf{X}_{(n)}$ 为张量沿第 n 个模态展开后的矩阵, $K^{(n)}$ 为其余因子的 Khatri-Rao 积, 则更新为

$$U^{(n)} \leftarrow \mathbf{X}_{(n)} K^{(n)} \left((K^{(n)})^\top K^{(n)} \right)^\dagger. \quad (3)$$

更新后对因子列归一化，并将尺度吸收到 λ 中。

Algorithm 1 CP-ALS 分解

输入: 张量 $\mathcal{W}$, 目标秩 R , 迭代次数 T

输出: 权重 λ 与因子矩阵 A, B, C, D

- 1: 随机初始化各因子, 令 $\lambda \leftarrow 1$
 - 2: **for** $t = 1$ to T **do**
 - 3: **for** 每个模态 n **do**
 - 4: 计算其余因子的 Khatri-Rao 积 $K^{(n)}$
 - 5: 用最小二乘更新 $U^{(n)}$
 - 6: 归一化因子列并更新 λ
 - 7: **end for**
 - 8: **end for**
-

3.2 微调策略

CP 替换后模型性能会下降，因此需要分解后的适应过程。本文比较全局微调、迭代式逐层微调和数据感知逐层微调三种策略。

3.2.1 全局微调

全局微调是最直接的策略：先一次性分解并替换所有目标卷积层，再用较小学习率对整个模型做分类微调。其流程见算法 2。该方法实现简单，但网络需要同时适应多个层的结构变化。

Algorithm 2 全局微调策略

输入: 预训练模型 M , 目标层列表 L , 秩 R

输出: 压缩并微调后的模型 M'

- 1: $M' \leftarrow M$
 - 2: **for** 每个目标层 $\ell \in L$ **do**
 - 3: 使用 CP-ALS 分解 ℓ 的权重
 - 4: 用四层 CP 结构替换 ℓ
 - 5: **end for**
 - 6: 对 M' 做全局分类微调
-

3.2.2 迭代式逐层微调

实验发现，全局微调并不能完全恢复网络的性能，我们认为这可能是因为多个层同时被替换为低秩结构后，每层引入的近似误差会在前向传播中叠加，梯度信号需要同时补偿所有层的偏差，优化难度随层数增加而上升。基于此，我们提出迭代式逐层微调方法：按深度逆序依次替换卷积层，优先训练更深层，每替换一层即立即进行分类微调，使网络在引入下一层低秩近似之前先适应当前的结构变化。等到所有层都替换完成后，再进行一次全局微调以消除残余误差。

3.2.3 数据感知逐层微调

现有方法还存在一处优化目标的错位：CP-ALS 最小化的是权重的 Frobenius 重构误差，而非该层在真实样本上的输出误差。由于权重接近并不等价于输出接近，重构误差小的分解仍可能改变该层的输出行为。为此，数据感知逐层微调在每次替换后增加一个激活重构步骤，固定其余层参数，用真实训练数据驱动 CP 层，直接最小化替换前后该层在相同输入特征图上的输出差异。

设第 ℓ_k 层替换前的卷积映射为 f_k^{ref} 、替换后的 CP 映射为 f_k^{CP} ，激活重构目标为

$$\min_{\ell_k} \mathbb{E}_{x \sim \mathcal{D}} \left\| f_k^{\text{CP}}(x_{\ell_k}) - f_k^{\text{ref}}(x_{\ell_k}) \right\|_F^2, \quad (4)$$

其中 x_{ℓ_k} 为当前模型（已部分替换）对输入 x 前向传播至 ℓ_k 时的输入特征图。完成激活重构后再进行分类微调，使每层的替换在函数层面而非权重层面得到充分补偿。该策略的整体流程见算法 3。

Algorithm 3 数据感知逐层微调策略

输入： 预训练模型 M ，目标卷积层列表 $L = (\ell_1, \dots, \ell_K)$ （按深度升序排列），秩 R

输出： 压缩并微调后的模型 M'

- 1: $M' \leftarrow M$ （复制全部权重）
 - 2: **for** $k = K, K-1, \dots, 1$ （按深度逆序遍历） **do**
 - 3: $f_k^{\text{ref}} \leftarrow$ 当前 M' 中层 ℓ_k 的卷积算子
 - 4: 对 f_k^{ref} 的权重做秩- R CP-ALS 分解，将 M' 中 ℓ_k 原地替换为四层 CP 结构
 - 5: **激活重构：** 固定 M' 中 ℓ_k 以外的所有参数，仅优化 ℓ_k 的 CP 权重，
以训练集数据驱动，迭代 T_{rec} 个 epoch 最小化 $\min_{\ell_k} \mathbb{E}_{x \sim \mathcal{D}} \left\| f_k^{\text{CP}}(x_{\ell_k}) - f_k^{\text{ref}}(x_{\ell_k}) \right\|_F^2$ ，
其中 x_{ℓ_k} 为当前 M' 对输入 x 前向传播至 ℓ_k 时的输入特征图
 - 6: **分类微调：** 以交叉熵损失对 M' 全参数做 T_{layer} 个 epoch 的 Adam 微调
 - 7: **end for**
 - 8: **最终全局微调：** 以交叉熵损失对 M' 全参数再做 T_{final} 个 epoch 的完整微调
-

4 数值结果

4.1 实验设置

所有主实验使用 5 个随机种子，报告均值 $\pm$ 标准误差。基线模型训练 10 epoch，batch size 为 64，CP-ALS 迭代 50 次。LeNet-5 使用秩 $R = 8$ ，SmallCNN 使用秩 $R = 16$ 。逐层微调策略中，激活重构迭代次数 $T_{\text{rec}} = 1$ ，每层分类微调轮数 $T_{\text{layer}} = 2$ ，最终全局微调轮数 $T_{\text{final}} = 5$ ；全局微调策略亦采用 5 个 epoch。所有微调阶段均使用 Adam 优化器，逐层分类微调与激活重构学习率为 10^{-3} ，全局微调学习率为 10^{-4} 。

4.2 性能指标

本文使用三类指标评估压缩效果：测试集 Top-1 准确率、目标卷积层参数量，以及整网单次前向传播所需 FLOPs。参数统计包含偏置项；参数压缩比和 FLOPs 降低比例均相对于对应基线模型计算。

表 1: 压缩统计

模型	秩	卷积参数	压缩比	基线模型 FLOPs	FLOPs 降低
LeNet-5	8	2572 $\rightarrow$ 414	6.21 $\times$	852.16 K	51.44%
SmallCNN	16	23296 $\rightarrow$ 2976	7.83 $\times$	15619.97 K	77.50%

4.3 结果与分析

表 2: 不同恢复策略的准确率对比 (% , 5 seed, 均值 $\pm$ 标准误差)

数据集	模型	基线模型	CP 无微调	Global FT	Layer-wise FT	Data-aware FT
MNIST	LeNet-5	98.89 $\pm$ 0.06	9.21 $\pm$ 0.84	96.95 $\pm$ 0.24	98.77 $\pm$ 0.07	99.02 $\pm$ 0.02
MNIST	SmallCNN	99.08 $\pm$ 0.06	10.79 $\pm$ 0.62	97.96 $\pm$ 0.14	99.18 $\pm$ 0.03	99.33 $\pm$ 0.02
Fashion-MNIST	LeNet-5	90.17 $\pm$ 0.14	9.59 $\pm$ 0.89	83.89 $\pm$ 0.71	89.39 $\pm$ 0.11	90.31 $\pm$ 0.08
Fashion-MNIST	SmallCNN	92.29 $\pm$ 0.07	9.49 $\pm$ 0.28	87.06 $\pm$ 0.45	92.03 $\pm$ 0.04	92.43 $\pm$ 0.09

CP 无微调。直接替换在所有设置下准确率均接近随机猜测 ($\approx 10\%$), 标准误差较大 (LeNet-5 可达 $\pm 0.89\%$), 说明 CP-ALS 所优化的权重重构误差与网络函数的保持之间存在根本性断层: 低秩近似在 Frobenius 范数意义下可以做到较小, 但叠加激活函数和多层传播后, 输出分布的偏移足以使分类完全失效。

全局微调。全局微调在 MNIST 上恢复效果尚可 (LeNet-5: 96.95%, SmallCNN: 97.96%), 但在更难的 Fashion-MNIST 上出现明显的精度损失 (LeNet-5 与基线差距达 6.28%, SmallCNN 差距达 5.23%), 且标准误差显著偏大 (LeNet-5 $\pm 0.71\%$), 表明优化过程不稳定。这印证了我们此前的假设: 多层同时被替换后, 梯度信号需同时补偿所有层的近似误差, 优化路径更难收敛。

逐层微调。迭代式逐层微调在所有设置下均大幅优于全局微调。MNIST 上两个模型均接近或超过基线 (LeNet-5: 98.77%, SmallCNN: 99.18%), Fashion-MNIST 上差距缩小至 1% 以内。标准误差相比全局微调明显下降, 说明逐层分解带来了更稳定的优化轨迹。这验证了“将多层联合适应问题分解为逐层单任务”的有效性。

数据感知逐层微调。在逐层微调基础上加入激活重构后, 四组实验均达到或略超基线模型, 其中 MNIST/SmallCNN 甚至高于基线 (99.33% vs 99.08%), 标准误差进一步压缩 (最低达 $\pm 0.02\%$)。精度提升的来源在于激活重构将 CP 分解的适应目标从权重空间的 Frobenius 误差转移到了真实输入特征图上的输出误差, 使每层替换后的函数行为更接近原始稠密层, 为后续分类微调提供了更好的初始点。上述四组实验同时保持表 1 中的参数压缩比和 FLOPs 降低。

5 结论

本文研究了基于 CP 分解的 CNN 卷积层压缩。实验表明, CP 分解可以显著降低卷积参数量与 FLOPs, 但朴素替换会严重破坏模型精度; 全局微调虽能恢复部分性能, 但面对多层同时替换时恢复能力有限。逐层微调通过逐层引入结构变化, 使模型逐步适应低秩卷积结构; 进一步加入数据感知激活重构后, 模型在 MNIST 与 Fashion-MNIST 上均基本恢复基线模型的准确率。

因此, 基于 CP 分解的卷积层压缩不仅取决于张量近似本身, 也取决于替换后的适应策略。对本实验规模而言, 逐层微调和层输出重构是比单纯全局微调更稳定的精度恢复方案。后续可在更大规模网络和数据集上继续验证该结论, 并研究每层秩的自动选择。

6 分工与贡献

本项目由两人合作完成, 分工如下:

- 周初平: 负责 CP-ALS 算法实现、全局微调和迭代式逐层微调的实验设计与实现, 贡献度: 0.5。
- 陈驰: 负责数据感知逐层微调的算法设计与实现, 以及所有实验结果的统计与分析, 贡献度: 0.5。

参考文献

- [1] Y. LeCun, Y. Bengio, and G. Hinton, “Deep learning,” *Nature*, vol. 521, no. 7553, pp. 436–444, May 2015.
- [2] T. G. Kolda and B. W. Bader, “Tensor decompositions and applications,” *SIAM Review*, vol. 51, no. 3, pp. 455–500, Aug. 2009.
- [3] P. Comon, “Tensors: A brief introduction,” *IEEE Signal Processing Magazine*, vol. 31, no. 3, pp. 44–53, May 2014.
- [4] Y. Panagakis, J. Kossaifi, G. G. Chrysos, J. Oldfield, M. A. Nicolaou, A. Anandkumar, and S. Zafeiriou, “Tensor methods in computer vision and deep learning,” *Proceedings of the IEEE*, vol. 109, no. 5, pp. 863–890, May 2021.
- [5] E. L. Denton, W. Zaremba, J. Bruna, Y. LeCun, and R. Fergus, “Exploiting linear structure within convolutional networks for efficient evaluation,” in *Proc. Advances in Neural Information Processing Systems (NIPS)*, 2014.
- [6] V. Lebedev, Y. Ganin, M. Rakhuba, I. Oseledets, and V. Lempitsky, “Speeding-up convolutional neural networks using fine-tuned CP-decomposition,” in *Proc. International Conference on Learning Representations (ICLR)*, 2015.